

Design Chess

Microsoft · Salesforce · Adobe · Amazon · Wells Fargo – ఇది ఒక ఆట design చేయడం గురించి కాదు; ఇది నియమాలు ఎక్కడ ఉండాలి అనే దాని గురించి.

ఆరు రకాల పావులు – ఇది polymorphism కి ఒక పాఠ్యపుస్తక ఉదాహరణ. కానీ నిజమైన కష్టం అక్కడ లేదు. ఇక్కడ మూడో విరుపు lo ఒక bishop కి తొమ్మిది కదలికలు ఉన్నాయి, మరియు వాటిలో సున్నా చెల్లుబాటు అవుతాయి – పావు నియమాల్లో ఏ తప్పు లేకుండా.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడ మనం **నిజంగా అడగలిగే** ఒక chess engine కడతాం – Fool's mate, castling, en passant, promotion, stalemate – అన్నీ నిజంగా నడిపి చూపిస్తాను.

CHESS ని INTERVIEW LO ఎందుకు అడుగుతారు

చాలా మంది అనుకుంటారు: "ఆరు పావులు, ఆరు subclasses – polymorphism, అయిపోయింది." అది **మొదటి పది నిమిషాలు** మాత్రమే, మరియు దాన్ని అందరూ చేస్తారు.

అసలు పరీక్ష తర్వాత వస్తుంది: **ఒక కదలిక "చెల్లుతుంది" అని ఎవరు నిర్ణయిస్తారు?** పావునా? Board నా? Game నా? ఎందుకంటే ఒక bishop తన నియమాల ప్రకారం కదలగలదు, కానీ ఆ కదలిక **మీ రాజుని చంపిస్తుంది** – అప్పుడు అది చట్టవిరుద్ధం.

అంటే "చెల్లుబాటు" అనేది పావుకి తెలియని విషయం. ఆ ఒక్క గుర్తింపే ఈ problem యొక్క కేంద్రం, మరియు §8 lo దాన్ని కళ్ళతో చూద్దాం.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Chess lo నిజమైన కష్టం ఎక్కడ ఉంది
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: పావుకి board కనిపించదు

3. Step – ఒక Piece class, ఒక switch
4. **మొదటి విరుపు** – `canMove(from, to)` తప్పు ప్రశ్న అడుగుతోంది
5. Step – ప్రతి పావు తన కదలికలని తయారు చేయడం

Part 3 – రెండో విరుపు: చెల్లుబాటు అనేది పావుది కాదు

6. **రెండో విరుపు** – తొమ్మిది కదలికలు, సున్నా చెల్లుబాటు
7. Step – చేసి చూడటం, ఆపై వెనక్కి తీయడం
8. `#simulate` ఎందుకు ఇంత జాగ్రత్తగా ఉండాలి

Part 4 – మూడో విరుపు: నియమాలకి చరిత్ర కావాలి

9. **మూడో విరుపు** – castling మరియు en passant board lo లేవు
10. Step – Castling · మూడు షరతులు
11. Step – En passant · ఒక్క కదలిక మాత్రమే గుర్తుండాలి
12. Step – Promotion · ఒక పావు వేరే పావుగా మారడం

Part 5 – పూర్తి system

13. మొత్తం code ఒకే చోట
14. నడిపి చూద్దాం – Fool's mate, castling, stalemate

Part 6 – Interview lo

15. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
16. నోటితో చెప్పాల్సిన English script
17. Follow-ups – undo, AI, performance
18. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Chess lo నిజమైన కష్టం ఎక్కడ ఉంది

"Design a chess game."

Chess నియమాలు అందరికీ తెలుసు, కాబట్టి ఇది సులభం అనిపిస్తుంది. కానీ ఆ నియమాలని **ఎక్కడ పెట్టాలి** అనేదే మొత్తం problem, మరియు అక్కడ మూడు పొరలు ఉన్నాయి:

"ఈ కదలిక చెల్లుతుందా?" – మూడు వేర్వేరు ప్రశ్నలు

ఒక్క ప్రశ్న అనిపిస్తుంది – నిజానికి మూడు, మరియు అవి వేర్వేరు చోట్ల ఉండాలి

1 · పావు నియమం

"Bishop ఏకర్థంగా కదులుతుంది"
ఇది పావుకి తెలిసిన విషయం
Board అవసరం లేదు

→ §5 · Polymorphism

2 · Board నియమం

"దారిలో ఎవరో ఉన్నారు"
"గమ్యంలో నా సొంత పావు"
పావు + board కలిసి

→ §5 · పావు board ని చూస్తుంది

3 · ఆట నియమం

"ఇది నా రాజుని బయటపెడుతుంది"
"Castling కి చరిత్ర కావాలి"
పావుకి ఇది తెలియదు · తెలియకూడదు

→ §7, §9 · ఇదే అసలు కష్టం

ఎవరు ఎక్కడ ఆగిపోతారు

పొర 1 ని అందరూ చేస్తారు – ఆరు subclasses, అది స్పష్టం
పొర 2 ని చాలా మంది చేస్తారు – కానీ **తప్పు signature** తో మొదలుపెట్టి ఇరుక్కుంటారు (§4)
పొర 3 ని **అడిగితేనే** గుర్తిస్తారు – మరియు అప్పటికి design మార్చడం కష్టం
మీరు మొదటి ఐదు నిమిషాల్లో ఈ మూడు పొరలని పేరుపెట్టి విడదీస్తే –
మిగతా interview అంతా సులభంగా నడుస్తుంది. అదే ఈ doc నేర్పేది.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
పూర్తి నియమాలు కావాలా – castling, en passant, promotion?	ఇది అతి ముఖ్యమైన ప్రశ్న. ఈ మూడు నియమాలకీ చరిత్ర కావాలి (§9)
Checkmate, stalemate గుర్తించాలా?	అవును అంటే – "ఈ ఆటగాడికి ఏ చట్టబద్ధ కదలిక లేదా?" అని అడగాలి
Undo కావాలా?	అవును అంటే ప్రతి కదలిక ఒక object కావాలి (Command)
AI ఆడాలా?	అవును అంటే <code>move</code> / <code>unmove</code> వేగంగా ఉండాలి – అదే §7
Draw నియమాలు (50-move, repetition)?	ఇవి చరిత్ర మీద ఆధారపడతాయి; నేను వీటిని scope బయట పెడతాను
Move notation (PGN) కావాలా?	ఒక వేరే పొర – core design ని మార్చదు
Timer/clock?	పూర్తిగా వేరే విషయం; scope బయట

"పూర్తి నియమాలు కావాలా?" – ఈ ఒక్క ప్రశ్న **DESIGN** ని రెండుగా చీలుస్తుంది

"కేవలం మౌలిక కదలికలు" అయితే – ఒక board, ఆరు పావు classes, అంతే. 20 నిమిషాల పని.

"పూర్తి నియమాలు" అయితే – castling కి "రాజు, rook ఇంతకుముందు కదలారా?" తెలియాలి. En passant కి "గత కదలిక ఏమిటి?" తెలియాలి. అంటే **board ఒక్కటే సరిపోదు** – ఒక `Game` కావాలి, మరియు దానికి చరిత్ర కావాలి.

ఈ ప్రశ్న అడగకపోతే మీరు మొదటి design రాసి, అపై దాన్ని తిరిగి రాయాలి. అడిగితే – మీరు మొదటి నుంచే సరైన ఆకారంలో ఉంటారు.

Part 2 – మొదటి విరుపు: పావుకి board కనిపించదు

3. Step – ఒక Piece class, ఒక switch

అతి సహజమైన మొదటి ప్రయత్నం: ఒక `Piece`, ఒక `type`, మరియు ఒక పెద్ద `switch`:

```
class Piece {
  constructor(type, colour) { Object.assign(this, { type, colour }); }

  canMove(from, to) {
    const dr = to.r - from.r, dc = to.c - from.c;
    const adr = Math.abs(dr), adc = Math.abs(dc);
    switch (this.type) {
      case 'ROOK': return dr === 0 || dc === 0;
      case 'BISHOP': return adr === adc;
      case 'QUEEN': return dr === 0 || dc === 0 || adr === adc;
      case 'KING': return adr <= 1 && adc <= 1;
      case 'KNIGHT': return (adr === 2 && adc === 1) || (adr === 1 && adc === 2);
      case 'PAWN': return dc === 0 && dr === (this.colour === 'W' ? -1 : 1);
      default: throw new Error('UNKNOWN_PIECE: ' + this.type);
    }
  }
}
```

```
Rook a1→a8 : true
Rook a1→h8 : false
Knight b1→c3: true
Knight b1→b3: false
```

నాలుగూ సరైన జవాబులు. Rook నిలువుగా/అడ్డంగా, knight L ఆకారంలో. గణితం సరిగ్గా ఉంది.

మరి సమస్య ఏమిటి?

4. మొదటి విరుపు – `canMove(from, to)` తప్పు ప్రశ్న అడుగుతోంది

```
సమస్య 1 – దారిలో ఎవరైనా ఉంటే?
Rook a1→a8 "చెల్లుతుంది": true
కానీ a4 లో ఒక piece ఉంటే? – canMove కి board గురించి తెలియదు.

సమస్య 2 – గమ్యంలో నా సొంత piece ఉంటే?
Rook a1→a2 "చెల్లుతుంది": true
```

కానీ a2 lo నా సొంత pawn ఉంది. అది తినలేను.

సమస్య 3 – కొత్త piece చేర్చాలంటే?

Archbishop (bishop+knight) కావాలంటే → ఈ switch ని తెరిచి మార్చాలి.

మొదటి విరుపు – తప్పు SIGNATURE

మూడు సమస్యల్లో మొదటి రెండూ ఒకే మూలం నుంచి వస్తున్నాయి: `canMove(from, to)` లో **board** లేదు.

Chess lo ఏ కదలికా board ని చూడకుండా నిర్ణయించలేం. Rook a1 నుంచి a8 కి వెళ్ళగలదా? – **దారిలో ఏముందో బట్టి**. ఆ సమాచారం లేకుండా జవాబు చెప్పడం అసాధ్యం.

మూడో సమస్య వేరే స్వభావం: `switch` అంటే **ప్రతి కొత్త పావుకీ ఉన్న code ని తెరవాలి**. ఇది Open/Closed ఉల్లంఘన, మరియు Deep Dive 01 §10 lo pricing తో చూసిన అదే నమూనా.

రెండు తప్పులూ ఒకే మార్పుతో పోతాయి: ప్రశ్నని తిప్పడం.

ప్రశ్నని తిప్పడం

తప్పు ప్రశ్న: "ఈ పావు ఇక్కడి నుంచి అక్కడికి వెళ్ళగలదా?" – ఇది ప్రతి గమ్యానికీ ఒక్కొక్కటిగా అడుగుతుంది.

సరైన ప్రశ్న: "ఈ పావు ఈ board మీద ఎక్కడెక్కడికి వెళ్ళగలదు?"

ఈ మార్పు రెండు పనులు చేస్తుంది: board సహజంగా parameter గా వస్తుంది, మరియు దారి అడ్డగించడం/సొంత పావు రెండూ **generation లోపలే** పరిష్కారమవుతాయి – ఒక్క `if` కూడా అదనంగా అవసరం లేకుండా.

5. Step – ప్రతి పావు తన కదలికలని తయారు చేయడం

`switch` పోయి, ఆరు subclasses వచ్చాయి. మరియు **జారే పావులకి** (rook, bishop, queen) ఉమ్మడి logic base class lo:

```
class Piece {
  constructor(colour) { this.colour = colour; this.hasMoved = false; }

  // జారే pieces కి ఉమ్మడి: ఒక దిక్కులో అడ్డు వచ్చేవరకు వెళ్ళడం
  slide(b, f, dirs) {
    const out = [];
    for (const [dr, dc] of dirs) {
      let r = f.r + dr, c = f.c + dc;
      while (b.inside(r, c)) {
        const p = b.at(r, c);
        if (!p) { out.push({ r, c }); } // ఖాళీ → వెళ్ళొచ్చు
        else { if (p.colour !== this.colour) out.push({ r, c }); // శత్రువు → తినొచ్చు
              break; } // ఏదైనా ఉంటే ఆగాలి
        r += dr; c += dc;
      }
    }
    return out;
  }
}
```

```

    }
}

class Rook extends Piece { get letter(){return 'r';} moves(b,f){ return
this.slide(b,f,ROOK_DIRS); } }
class Bishop extends Piece { get letter(){return 'b';} moves(b,f){ return
this.slide(b,f,BISHOP_DIRS); } }
class Queen extends Piece { get letter(){return 'q';} moves(b,f){ return
this.slide(b,f,[...ROOK_DIRS,...BISHOP_DIRS]); } }

```

ఆ **slide** lo మూడు నియమాలూ ఒకే చోట ఉన్నాయి: ఖాళీ గడికి వెళ్ళొచ్చు, శత్రు పావుని తినొచ్చు (ఆపై ఆగాలి), సొంత పావు దగ్గర ఆగాలి. §4 యొక్క మొదటి రెండు సమస్యలూ ఇక్కడే పరిష్కారమయ్యాయి.

మరియు **Queen కి సొంత code లేదు** – అది కేవలం rook దిక్కులు + bishop దిక్కులు. ఇది ఒక చిన్న, అందమైన వివరం.

నడిపి చూద్దాం – a1 తెల్ల rook, a2 తెల్ల knight, a4 నల్ల rook:

```

8 . . . . . . . .
4 r . . . . . . .
2 N . . . . . . .
1 R . . . . . . .
  a b c d e f g h

a1 Rook కదలికలు: b1 c1 d1 e1 f1 g1 h1
→ a2 lo సొంత knight ఉంది కాబట్టి పైకి వెళ్ళలేదు
→ b1..h1 ఖాళీ కాబట్టి అటు వెళ్ళగలదు

a4 నల్ల Rook కదలికలు: a3 a2 a5 a6 a7 a8 b4 c4 d4 e4 f4 g4 h4
→ a2 వద్ద తెల్ల knight ని *తినగలదు*, అక్కడే ఆగుతుంది

```

తెల్ల rook **పైకి అస్సలు వెళ్ళలేదు** (సొంత knight). నల్ల rook **a2 వరకు వెళ్ళి ఆగింది** (శత్రు knight ని తినగలదు, కానీ దాన్ని దాటలేదు). **మొదటి విరుపు పూర్తిగా సరిచేయబడింది.**

Part 3 – రెండో విరుపు: చెల్లుబాటు అనేది పావుది కాదు

6. రెండో విరుపు – తొమ్మిది కదలికలు, సున్నా చెల్లుబాటు

ఇప్పుడు ఒక సాధారణ, కానీ నిర్ణయాత్మకమైన స్థానం. తెల్ల రాజు **e1**, తెల్ల bishop **e2**, నల్ల rook **e8**:

```
8 . . . . r . . .
2 . . . . B . . .
1 . . . . K . . .
  a b c d e f g h
```

e2 Bishop "చేయగల" కదలికలు: f1 d1 f3 g4 h5 d3 c4 b5 a6

తొమ్మిది కదలికలు, మరియు మన `slide` ప్రకారం అన్నీ సరైనవే – bishop వికర్ణంగా కదులుతోంది, దారిలో ఎవరూ లేరు.

కానీ ఆలోచించండి: bishop **e**-వరుస నుంచి తప్పుకుంటే ఏమవుతుంది?

రెండో విరుపు – తొమ్మిదింటిలో ఒక్కటే చెల్లదు

Bishop e2 నుంచి ఎక్కడికి వెళ్ళినా, అది e-వరుసని వదిలేస్తుంది. అప్పుడు **e8** నల్ల rook కి **e1** తెల్ల రాజు వరకు దారి తెరుచుకుంటుంది – అంటే మన రాజు check lo పడతాడు.

Chess నియమం: **మీ సొంత రాజుని check lo పెట్టే కదలిక చట్టవిరుద్ధం**. కాబట్టి ఆ తొమ్మిదింటిలో ఒక్కటే చెల్లదు. Bishop పూర్తిగా కట్టుబడి ఉంది (pinned).

ఇక్కడ ముఖ్యమైనది: `Bishop.moves()` lo ఏ తప్పు లేదు. అది సరిగ్గానే పనిచేసింది. తప్పు ఏమిటంటే – చెల్లుబాటుని నిర్ణయించడం పావు పని కాదు.

మరియు bishop కి అది తెలియనూ కూడదు. దానికి రాజు ఎక్కడ ఉన్నాడో, శత్రువులు ఎక్కడ ఉన్నారో తెలిస్తే – అది ఇక ఒక bishop కాదు, అది మొత్తం ఆటని తెలుసుకున్న ఒక దేవుడు.

కాబట్టి రెండు రకాల కదలికలు ఉన్నాయి

	ఎవరు నిర్ణయిస్తారు	అర్థం
Pseudo-legal (ముడి)	<code>Piece.moves()</code>	"ఈ పావు నియమాల ప్రకారం ఇక్కడికి వెళ్ళగలదు"
Legal (చట్టబద్ధం)	<code>Game.legalMoves()</code>	"...మరియు అది నా రాజుని ప్రమాదంలో పెట్టదు"

ఈ రెండు పేర్లూ నిజమైన chess-engine పరిభాష. **Interview lo వీటిని వాడండి** – ఇది మీరు ఈ విభజనని తెలిసి చేస్తున్నారని చూపిస్తుంది.

7. Step – చేసి చూడటం, అపై వెనక్కి తీయడం

"ఈ కదలిక నా రాజుని check lo పెడుతుందా?" – దీన్ని ఎలా తెలుసుకోవాలి?

తెలివైన పద్ధతులు ఉన్నాయి (pin detection, ray casting). కానీ అత్యంత సరళమైనది, మరియు **తప్పు కాని** పద్ధతి: **ఆ కదలికని నిజంగా చేసి, రాజు క్షేమమా చూసి, వెనక్కి తీయడం.**

```
#simulate(from, to, fn) {
  const g = this.board.grid;
  const moving = g[from.r][from.c], captured = g[to.r][to.c];
  const prevEp = this.board.enPassant;

  g[to.r][to.c] = moving; g[from.r][from.c] = null;      // చేయడం
  // ... (en passant నిర్వహణ - §11) ...

  const result = fn();                                    // ప్రశ్న అడగడం

  g[from.r][from.c] = moving; g[to.r][to.c] = captured;  // వెనక్కి తీయడం
  this.board.enPassant = prevEp;
  return result;
}

legalMoves(from) {
  const p = this.board.at(from.r, from.c);
  if (!p || p.colour !== this.turn) return [];
  const pseudo = p.moves(this.board, from);
  return pseudo.filter((to) => this.#simulate(from, to, () =>
    !this.inCheck(p.colour)));
}
```

మరియు "check lo ఉన్నాడా?" అనేది ఆశ్చర్యకరంగా సులభం – **శత్రువుల ముడి కదలికల్లో రాజు గడి ఉందా?**

```
isAttacked(sq, byColour) {
  for (const { piece, at } of this.board.pieces(byColour))
    if (piece.moves(this.board, at).some((m) => m.r === sq.r && m.c === sq.c)) return
    true;
  return false;
}

inCheck(colour) {
  const k = this.board.find(colour, King);
  return k ? this.isAttacked(k, other(colour)) : false;
}
```

ఇక్కడ ఒక అందమైన పునర్నివయోగం ఉంది. "రాజు దాడిలో ఉన్నాడా?" అని తెలుసుకోవడానికి మనం **అదే moves()** ని వాడుతున్నాం – శత్రు పావుల మీద. కొత్త code ఏమీ లేదు.

మరియు ఇక్కడ **ముడి** కదలికలు వాడటం సరైనది, చట్టబద్ధమైనవి కాదు. ఎందుకంటే ఒక పావు కట్టుబడి ఉన్నా కూడా అది దాడి చేస్తూనే ఉంటుంది – శత్రు రాజు ఆ గడికి రాలేడు. ఇది chess యొక్క ఒక సూక్ష్మమైన నియమం, మరియు ముడి కదలికలు వాడటం వల్ల అది **ఉచితంగా** సరైనదవుతుంది.

అదే pinned స్థానంలో:

```
--- కట్టుబడిన (pinned) bishop ---  
Bishop యొక్క ముడి కదలికలు: 9  
చట్టబద్ధమైన కదలికలు : 0 ← అన్నీ రాజుని బయటపెడతాయి
```

9 → 0. రెండో విరుపు సరిచేయబడింది, మరియు Bishop class lo ఒక్క అక్షరం కూడా మారలేదు.

8. #simulate ఎందుకు ఇంత జాగ్రత్తగా ఉండాలి

ఆ method చిన్నది, కానీ ప్రతి పంక్తికీ ఒక కారణం ఉంది – మరియు ఒకటి మర్చిపోతే bugs నిశ్శబ్దంగా వస్తాయి.

పంక్తి	ఎందుకు • మర్చిపోతే ఏమవుతుంది
<code>captured</code> ని గుర్తుపెట్టుకోవడం	వెనక్కి తీసేటప్పుడు తిన్న పావుని తిరిగి పెట్టాలి. లేకపోతే simulation ప్రతిసారీ ఒక పావుని తినేస్తుంది
<code>prevEp</code> ని గుర్తుపెట్టుకోవడం	En passant గడి కదలికతో మారుతుంది; తిరిగి పెట్టకపోతే తర్వాతి కదలికలు తప్పుగా లెక్కించబడతాయి
En passant ని ప్రత్యేకంగా	అక్కడ తిన్న పావు గమ్యం గడిలో ఉండదు – అది పక్కన ఉంటుంది (§11)
కొత్త object సృష్టించకపోవడం	Board ని copy చేస్తే సులభం, కానీ చాలా నెమ్మది – కింద చూడండి

Board ని copy చేయొచ్చు కదా?

చేయొచ్చు, మరియు అది సులభం:

```
const copy = structuredClone(this.board); // ✗ సులభం, కానీ ఖరీదు
```

కానీ ఆలోచించండి: `legalMoves` ప్రతి కదలికకీ ఒకసారి `simulate` చేస్తుంది. `status()` ప్రతి పావు యొక్క ప్రతి కదలికకీ. ఒక్క `status()` పిలుపు **వందల simulations**. Copy చేస్తే అది వందల board copies.

Make/unmake అనే ఈ నమూనా ప్రతి నిజమైన chess engine వాడేదే, సరిగ్గా ఈ కారణానికే. AI చేర్చాలంటే (§17) ఇది **లక్షల** సార్లు నడుస్తుంది, మరియు అప్పుడు ఈ ఎంపిక తేడా చేస్తుంది.

Part 4 – మూడో విరుపు: నియమాలకి చరిత్ర కావాలి

9. మూడో విరుపు – castling మరియు en passant board lo తేవు

ఇప్పటిదాకా ప్రతి నిర్ణయం ఇప్పటి board ని చూసి తీసుకున్నాం. కానీ chess lo మూడు నియమాలు ఆ మాదిరి కావు:

మూడు నియమాలు • BOARD ఒక్కటే సరిపోదు

ఒకే board స్థానం - కానీ జవాబు చరిత్రని బట్టి మారుతుంది

Castling రాజు, rook ఇద్దరూ ఇంతకుముందు కదలకుండా ఉండాలి రాజు e1, rook h1 - కానీ అవి కదిలి తిరిగివచ్చాయా? board చెప్పదు	En passant సరిగ్గా గత కదలిక ఏమిటి? శత్రు pawn ఇప్పుడే రెండు గడులు దూకిందా? ఒక్క కదలిక ఆలస్యమైనా ఈ హక్కు పోతుంది	Promotion ఒక pawn చివరి వరుసకి చేరితే - అది వరే పావుగా మారుతుంది చరిత్ర అవసరం లేదు, కానీ
---	--	---

ఇవి మూడూ ఒకే విషయం చెప్తున్నాయి

ఒక కదలిక "ఒక పావు ఇక్కడి నుంచి అక్కడికి" అని మాత్రమే కాదు.

Castling lo రెండు పావులు కదులుతాయి • en passant lo తిన్నది గమ్యంలో లేదు •

promotion lo పావే మారిపోతుంది.

అంటే Move ఒక tuple కాదు - అది ఒక object, మరియు Game దాన్ని అమలు చేయాలి.

మన design ఇప్పటికే సగం సిద్ధంగా ఉంది: Piece lo hasMoved ఉంది (castling కి), మరియు Board lo enPassant ఉంది. ఇప్పుడు వాటిని వాడదాం.

10. Step – Castling • మూడు షరతులు

Castling కి మూడు వేర్వేరు షరతులు, మరియు అవి మూడు వేర్వేరు చోట్ల నుంచి వస్తాయి:

```
#castlingMoves(from, king) {
  const out = [], row = from.r;
  for (const [rookCol, step, through] of [[7, 1, [5,6]], [0, -1, [3,2,1]]]) {
    const rook = this.board.at(row, rookCol);
    if (!(rook instanceof Rook) || rook.hasMoved || rook.colour !== king.colour)
      continue; // 1
    if (through.some((c) => this.board.at(row, c))) continue;
    // 2
    const cross = [from.c + step, from.c + 2*step];
    if (cross.some((c) => this.isAttacked({r:row,c}, other(king.colour)))) continue;
    // 3
    out.push({ r: row, c: from.c + 2*step, castle: true });
  }
}
```

```
return out;
}
```

1. **చరిత్ర** – రాజ, rook ఇద్దరూ కదలకుండా ఉండాలి (hasMoved).
2. **Board** – మధ్యలో ఉన్న గడులు ఖాళీగా ఉండాలి.
3. **దాడి** – రాజ దాటే గడులు దాడిలో ఉండకూడదు. (మరియు legalMoves lo ఇప్పటికే "రాజ check lo ఉండకూడదు" అని తనిఖీ చేశాం.)

మూడో షరత సూక్ష్మమైనది: రాజ e1 నుంచి g1 కి వెళ్ళేటప్పుడు **f1 గుండా** వెళ్ళాడు. F1 దాడిలో ఉంటే castling చెల్లదు – రాజ ఆగకపోయినా.

నడిపి చూద్దాం:

```
--- Castling ---
రాజ e1 కదలికలు: e2 f1 d1 f2 d2 g1(castle) c1(castle)
0-0 తర్వాత:
1 R . . . . R K .
a b c d e f g h

--- Castling ఎందుకు ఆగుతుంది ---
f-వరుసలో నల్ల rook → రాజ f1 దాటాలి, అది దాడిలో ఉంది
రాజ కదలికలు: e2 d1 d2
```

మొదటి భాగం: రెండు వైపులా castling అందుబాటులో, మరియు 0-0 తర్వాత board R R K .
– rook a1 lo ఉంది, రాజ g1, మరియు **h1 rook f1 కి వచ్చింది**. రెండు పావులూ ఒకే కదలికలో కదిలాయి.

రెండో భాగం: f-వరుసలో ఒక నల్ల rook పెడితే – f1, f2 **రెండూ** కదలికల జాబితా నుంచి పోయాయి (రాజ దాడిలోకి వెళ్ళలేడు), మరియు **castling కూడా పోయింది**. మూడో షరత పనిచేసింది.

11. Step – En passant • ఒక్క కదలిక మాత్రమే గుర్తుండాలి

En passant chess lo అత్యంత విచిత్రమైన నియమం: ఒక శత్రు pawn **రెండు గడులు దూకి** మీ pawn పక్కన ఆగితే, మీరు దాన్ని అది ఒక్క గడ కదిలినట్టు తీసుచ్చు – **కానీ వెంటనే మాత్రమే**.

దీనికి కావలసిన చరిత్ర **ఒక్క గడి**: "గత కదలికలో ఏ గడి దాటబడింది?"

```
// move() చివర్లో:
this.board.enPassant = (piece instanceof Pawn && Math.abs(to.r - from.r) === 2)
? { r: (from.r + to.r) / 2, c: from.c } // దాటిన గడి
: null; // ← మిగతా ప్రతి కదలికా దీన్ని తుడుస్తుంది
```

ఆ : **null** అత్యంత ముఖ్యమైన భాగం. ఏ ఇతర కదలికైనా ఆ హక్కుని **తక్షణం** తుడిచేస్తుంది – అదే "వెంటనే మాత్రమే" అనే నియమం.

మరియు **Pawn.moves()** lo:

```
else if (!t && b.enPassant && b.enPassant.r === r && b.enPassant.c === c)
    out.push({ r, c, enPassant: true }); // ← ఖాళీ గడికి "తినడం"
```

```
--- En passant ---
d7→d5 తర్వాత en-passant గడి: d6
e5 తెల్ల pawn కదలికలు: e6 d6(ep)
exd6 e.p. తర్వాత d5 lo ఏముంది? ఖాళీ ← నల్ల pawn పోయింది
```

చివరి పంక్తి గమనించండి. తెల్ల pawn **d6** కి వెళ్ళింది, కానీ తిన్న నల్ల pawn **d5** lo ఉంది. అంటే **తిన్న పావు గమ్యం గడిలో లేదు** – chess lo ఇది ఒక్క ఇక్కడే జరుగుతుంది, మరియు అందుకే `#simulate` lo దానికి ప్రత్యేక నిర్వహణ కావాలి (98).

12. Step – Promotion • ఒక పావు వేరే పావుగా మారడం

Pawn చివరి వరుసకి చేరితే అది queen/rook/bishop/knight గా మారుతుంది:

```
if (piece instanceof Pawn && (to.r === 0 || to.r === 7)) {
    g[to.r][to.c] = new promoteTo(piece.colour); // ← కొత్త object, పాతది పోతుంది
    g[to.r][to.c].hasMoved = true;
}
```

```
--- Promotion ---
a7→a8 తర్వాత a8 lo: Queen (Q)
```

ఇక్కడ ఒక design ఎంపిక ఉంది: pawn యొక్క *type* ని మార్చడం కాదు – **కొత్త object సృష్టించి పాతదాన్ని భర్తీ చేయడం.**

ఎందుకు? ఎందుకంటే §5 lo మన పావులు **ప్రవర్తన ద్వారా** వేరుపడతాయి, ఒక `type` field ద్వారా కాదు. `pawn.type = 'QUEEN'` అని రాయడం మనల్ని §3 యొక్క switch వైపు వెనక్కి లాగుతుంది. **Polymorphic design** lo **"రకం మారడం"** అంటే **"వస్తువు మారడం"**.

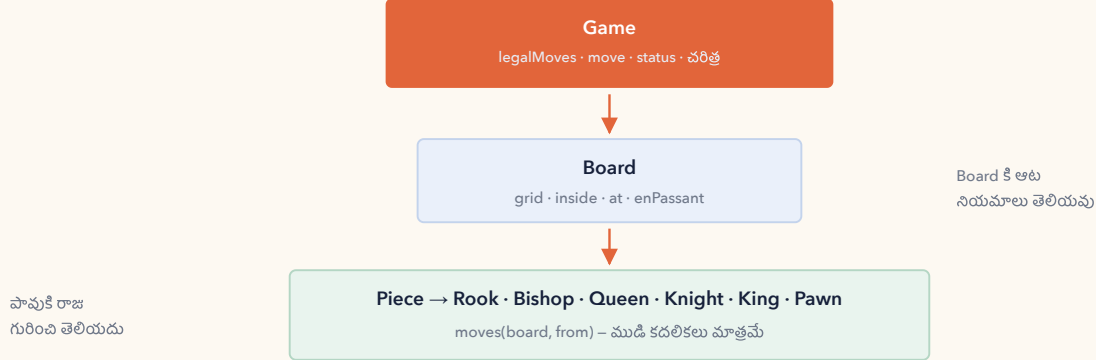
`promoteTo` ఒక parameter గా ఉండటం కూడా ముఖ్యం – **underpromotion** (queen కాకుండా knight ఎంచుకోవడం) నిజమైన ఆటల్లో జరుగుతుంది, మరియు అది ఒక hardcoded `Queen` తో సాధ్యం కాదు.

Part 5 – పూర్తి system

13. మొత్తం code ఒక చోట

నిర్మాణం · ఎవరికి ఏమి తెలుసు

మూడు పాఠాలు, మరియు ఒక స్పష్టమైన దిశ: పైవాళ్ళకి కిందివాళ్ళ గురించి తెలుసు, తిరగబడి కాదు



```
const W = 'W', B = 'B';
const other = (c) => (c === W ? B : W);

class Piece {
  constructor(colour) { this.colour = colour; this.hasMoved = false; }
  get symbol() { return this.colour === W ? this.letter.toUpperCase() : this.letter.toLowerCase(); }
  slide(b, f, dirs) {
    const out = [];
    for (const [dr, dc] of dirs) {
      let r = f.r + dr, c = f.c + dc;
      while (b.inside(r, c)) {
        const p = b.at(r, c);
        if (!p) out.push({ r, c });
        else { if (p.colour !== this.colour) out.push({ r, c }); break; }
        r += dr; c += dc;
      }
    }
    return out;
  }
  step(b, f, deltas) {
    return deltas.map(([dr, dc]) => ({ r: f.r + dr, c: f.c + dc }))
      .filter((s) => b.inside(s.r, s.c) && b.at(s.r, s.c)?.colour !== this.colour);
  }
}

const ROOK_DIRS = [[1,0],[-1,0],[0,1],[0,-1]];
const BISHOP_DIRS = [[1,1],[1,-1],[-1,1],[-1,-1]];
const KNIGHT_STEPS = [[2,1],[2,-1],[-2,1],[-2,-1],[1,2],[1,-2],[-1,2],[-1,-2]];
const KING_STEPS = [...ROOK_DIRS, ...BISHOP_DIRS];

class Rook extends Piece { get letter(){return 'r';} moves(b,f){ return
```

```

this.slide(b,f,ROOK_DIRS); } }
class Bishop extends Piece { get letter(){return 'b';} moves(b,f){ return
this.slide(b,f,BISHOP_DIRS); } }
class Queen extends Piece { get letter(){return 'q';} moves(b,f){ return
this.slide(b,f,[...ROOK_DIRS,...BISHOP_DIRS]); } }
class Knight extends Piece { get letter(){return 'n';} moves(b,f){ return
this.step(b,f,KNIGHT_STEPS); } }
class King extends Piece { get letter(){return 'k';} moves(b,f){ return
this.step(b,f,KING_STEPS); } }

class Pawn extends Piece {
  get letter(){ return 'p'; }
  moves(b, f) {
    const dir = this.colour === W ? -1 : 1;
    const start = this.colour === W ? 6 : 1;
    const out = [];
    if (b.inside(f.r+dir, f.c) && !b.at(f.r+dir, f.c)) {
      out.push({ r: f.r+dir, c: f.c });
      if (f.r === start && !b.at(f.r+2*dir, f.c)) out.push({ r: f.r+2*dir, c: f.c });
    }
    for (const dc of [-1, 1]) {
      const r = f.r+dir, c = f.c+dc;
      if (!b.inside(r,c)) continue;
      const t = b.at(r,c);
      if (t && t.colour !== this.colour) out.push({ r, c });
      else if (!t && b.enPassant && b.enPassant.r === r && b.enPassant.c === c)
        out.push({ r, c, enPassant: true });
    }
    return out;
  }
}

class Board {
  constructor() { this.grid = Array.from({length:8},()=>Array(8).fill(null));
this.enPassant = null; }
  inside(r,c){ return r>=0&&r<8&&c>=0&&c<8; }
  at(r,c){ return this.inside(r,c) ? this.grid[r][c] : null; }
  put(r,c,p){ this.grid[r][c]=p; return this; }
  find(colour, Type) {
    for (let r=0;r<8;r++) for (let c=0;c<8;c++) {
      const p=this.grid[r][c];
      if (p && p.colour===colour && p instanceof Type) return { r, c };
    }
    return null;
  }
  pieces(colour) {
    const out=[];
    for (let r=0;r<8;r++) for (let c=0;c<8;c++) {
      const p=this.grid[r][c];
      if (p && p.colour===colour) out.push({ piece:p, at:{r,c} });
    }
    return out;
  }
  show() {
    return this.grid.map((row,r)=>(8-r)+' '+row.map(p=>p?p.symbol:'.').join('
')).join('\n')
      + '\n a b c d e f g h';
  }
  static initial() {

```

```

    const b = new Board();
    const back = [Rook,Knight,Bishop,Queen,King,Bishop,Knight,Rook];
    back.forEach((T,c)=>{ b.put(0,c,new T(B)); b.put(7,c,new T(W)); });
    for (let c=0;c<8;c++) { b.put(1,c,new Pawn(B)); b.put(6,c,new Pawn(W)); }
    return b;
  }
}

class Game {
  #history = [];
  constructor(board = Board.initial(), turn = W) { Object.assign(this, { board, turn
}); }

  isAttacked(sq, byColour) {
    for (const { piece, at } of this.board.pieces(byColour))
      if (piece.moves(this.board, at).some((m) => m.r===sq.r && m.c===sq.c)) return
true;
    return false;
  }
  inCheck(colour) {
    const k = this.board.find(colour, King);
    return k ? this.isAttacked(k, other(colour)) : false;
  }

  #simulate(from, to, fn) {
    const g = this.board.grid;
    const moving = g[from.r][from.c], captured = g[to.r][to.c];
    const prevEp = this.board.enPassant;
    g[to.r][to.c] = moving; g[from.r][from.c] = null;
    let epCaptured = null, epSq = null;
    if (moving instanceof Pawn && prevEp && to.r===prevEp.r && to.c===prevEp.c &&
from.c!==to.c) {
      epSq = { r: from.r, c: to.c };
      epCaptured = g[epSq.r][epSq.c]; g[epSq.r][epSq.c] = null;
    }
    const result = fn();
    g[from.r][from.c] = moving; g[to.r][to.c] = captured;
    if (epSq) g[epSq.r][epSq.c] = epCaptured;
    this.board.enPassant = prevEp;
    return result;
  }

  legalMoves(from) {
    const p = this.board.at(from.r, from.c);
    if (!p || p.colour !== this.turn) return [];
    const pseudo = p.moves(this.board, from);
    const legal = pseudo.filter((to) => this.#simulate(from, to, () =>
!this.inCheck(p.colour)));
    if (p instanceof King && !p.hasMoved && !this.inCheck(p.colour))
      legal.push(...this.#castlingMoves(from, p));
    return legal;
  }

  #castlingMoves(from, king) {
    const out = [], row = from.r;
    for (const [rookCol, step, through] of [[7, 1, [5,6]], [0, -1, [3,2,1]]]) {
      const rook = this.board.at(row, rookCol);
      if (!(rook instanceof Rook) || rook.hasMoved || rook.colour !== king.colour)
continue;

```



```

        if (through.some((c) => this.board.at(row, c))) continue;
        const cross = [from.c + step, from.c + 2*step];
        if (cross.some((c) => this.isAttacked({r:row,c}, other(king.colour))))
continue;
        out.push({ r: row, c: from.c + 2*step, castle: true });
    }
    return out;
}

allLegalMoves(colour = this.turn) {
    const out = [];
    for (const { at } of this.board.pieces(colour))
        for (const to of this.legalMoves(at)) out.push({ from: at, to });
    return out;
}

move(from, to, promoteTo = Queen) {
    const chosen = this.legalMoves(from).find((m) => m.r===to.r && m.c===to.c);
    if (!chosen) return { ok: false, reason: 'ILLEGAL_MOVE' };

    const g = this.board.grid;
    const piece = g[from.r][from.c];
    const captured = g[to.r][to.c];
    g[to.r][to.c] = piece; g[from.r][from.c] = null;

    if (chosen.castle) {
        // rook కూడా కదులుతుంది
        const rookFrom = to.c > from.c ? 7 : 0, rookTo = to.c > from.c ? to.c-1 :
to.c+1;
        g[to.r][rookTo] = g[to.r][rookFrom]; g[to.r][rookFrom] = null;
        g[to.r][rookTo].hasMoved = true;
    }
    if (piece instanceof Pawn && chosen.enPassant) g[from.r][to.c] = null;
    if (piece instanceof Pawn && (to.r === 0 || to.r === 7)) {
        g[to.r][to.c] = new promoteTo(piece.colour);
        g[to.r][to.c].hasMoved = true;
    }

    this.board.enPassant = (piece instanceof Pawn && Math.abs(to.r-from.r) === 2)
        ? { r: (from.r+to.r)/2, c: from.c } : null;
    piece.hasMoved = true;
    this.#history.push({ from, to, piece, captured });
    this.turn = other(this.turn);
    return { ok: true, status: this.status() };
}

status() {
    const hasMoves = this.allLegalMoves(this.turn).length > 0;
    if (hasMoves) return this.inCheck(this.turn) ? 'CHECK' : 'IN_PROGRESS';
    return this.inCheck(this.turn) ? 'CHECKMATE' : 'STALEMATE';
}
}

```

status() – నాలుగు స్థితులు, రెండు ప్రశ్నలు

ఆ చివరి method చిన్నది కానీ చక్కనిది. **రెండు boolean ప్రశ్నలు** నాలుగు స్థితులని ఇస్తాయి:

	కదలికలు ఉన్నాయి	కదలికలు లేవు
Check lo ఉన్నాడు	CHECK	CHECKMATE
Check lo లేడు	IN_PROGRESS	STALEMATE

Checkmate మరియు stalemate మధ్య తేడా **ఒక్కటే** – "రాజు ఇప్పుడు దాడిలో ఉన్నాడా?". చాలా మంది వీటికి వేర్వేరు logic రాస్తారు; ఇది అనవసరం.

14. నడిపి చూద్దాం – Fool's mate, castling, stalemate

Fool's mate – chess lo అత్యంత వేగవంతమైన checkmate, రెండు కదలికల్లో:

```
f2→f3: IN_PROGRESS
e7→e5: IN_PROGRESS
g2→g4: IN_PROGRESS
d8→h4: CHECKMATE
8 r n b . k b n r
7 p p p p . p p p
6 . . . . .
5 . . . . p . .
4 . . . . . P q
3 . . . . . P .
2 P P P P P . . P
1 R N B Q K B N R
a b c d e f g h
```

నల్ల queen **h4** కి వచ్చింది. అది e1 తెల్ల రాజుని వికర్ణంగా దాడి చేస్తోంది (h4-g3-f2-e1), మరియు **f2, g2 pawns కదిలిపోయాయి** కాబట్టి ఆ దారి తెరిచి ఉంది. తెల్లవాడికి ఏ కదలికా లేదు → **CHECKMATE**. మన `status()` దాన్ని సరిగ్గా గుర్తించింది.

Stalemate – మరియు ఇది checkmate కి పూర్తి వ్యతిరేకం:

```
8 k . . . . .
6 . Q . . . . .
1 . . K . . . .
a b c d e f g h
నల్ల రాజు check lo ఉన్నాడా? false
నల్లవాడికి చట్టబద్ధమైన కదలికలు: 0
స్థితి: STALEMATE ← check కాదు, కానీ కదలిక లేదు

--- పోల్చి చూడటం: అదే స్థానం, queen b7 కి జరిపితే ---
check lo ఉన్నాడా? true | కదలికలు: 1 | స్థితి: CHECK
```

ఈ రెండు పంక్తులని పక్కపక్కన చూడండి

Queen **b6** lo – నల్ల రాజు a8 check lo **లేడు** (b6 నుంచి a8 కి దారి లేదు), కానీ a7, b7, b8 మూడూ దాడిలో ఉన్నాయి. కదలిక లేదు → **STALEMATE** (ఆట డ్రా).

Queen **b7** కి జరిపితే – ఇప్పుడు రాజు check lo ఉన్నాడు, మరియు అతను queen ని తినగలడు (b7 రాజు పక్కనే ఉంది, తెల్ల రాజు c1 దాన్ని కాపాడలేదు) → **CHECK**, ఒక కదలికతో.

ఒక్క గడి తేడా. ఒకటి డ్రా, ఇంకొకటి ఆట కొనసాగడం. మన రెండు-ప్రశ్నల **status()** రెండింటినీ సరిగ్గా వేరుచేసింది, ప్రత్యేక code లేకుండా.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>Piece</code> , <code>colour</code>	మౌలిక భావన
§4 (ఎరుపు)	<code>moves(board, from)</code> – switch కాదు	Board లేకుండా జవాబు చెప్పలేం
§5	<code>slide</code> , <code>step</code> , ఆరు subclasses	దారి అడ్డగింపు + capture ఒకే చోట
§6 (ఎరుపు)	<code>Game.legalMoves()</code>	చెల్లుబాటు పావు పని కాదు
§7	<code>#simulate</code> , <code>isAttacked</code> , <code>inCheck</code>	చేసి చూసి వెనక్కి తీయడం
§9 (ఎరుపు)	<code>hasMoved</code> , <code>enPassant</code>	కొన్ని నియమాలకి చరిత్ర కావాలి
§10-12	Castling, en passant, promotion	<code>Move</code> ఒక tuple కాదు
ఇక్కడ	<code>status()</code>	రెండు ప్రశ్నలు → నాలుగు స్థితులు

Part 6 – Interview Io

15. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Chess Io code ఎక్కువ – కాబట్టి ఏది రాయాలో, ఏది చెప్పాలో ఎంచుకోవడం కీలకం

5 నిమి

Clarify – "పూర్తి నియమాల్లా?" తప్పనిసరి ప్రశ్న

6 నిమి

మూడు పొరలని పేరుపెట్టడం (§1) · Piece hierarchy

10 నిమి – moves()

slide + step · rook/bishop/knight చాలు

14 నిమి – చట్టబద్ధత

ఇదే interview యొక్క కేంద్రం · pin, simulate

10 నిమి

Castling/en passant · status() · follow-ups

ఏమి తప్పక చెప్పాలి

1. **మూడు పొరలు** (§1) – పావు నియమం / board నియమం / ఆట నియమం. ఇది మీ మొదటి వాక్యం.
2. **Pin** (§6) – ఒక ఉదాహరణ గీసి, "తొమ్మిది కదలికలు, సున్నా చెల్లుబాటు" అని చూపించండి.
3. **Make/unmake, copy కాదు** (§8) – మరియు ఎందుకే.
4. **Castling/en passant కి చరిత్ర కావాలి** (§9) – ఇది మీరే లేవనెత్తండి.
5. **Checkmate vs stalemate ఒకే logic** (§13) – రెండు ప్రశ్నలు, నాలుగు స్థితులు.

ఏమి వదిలేయాలి

- **ఆరు పావులూ రాయొద్దు** – `slide` చూపించి, rook + bishop రాసి, "queen రెండూ కలిపి, knight/king step" అని చెప్పండి.
- **Castling code పూర్తిగా రాయొద్దు** – మూడు షరతులని చెప్పండి, అది సరిపోతుంది.
- `Board.initial()` వదిలేయండి – ఇది కేవలం setup.

16. నీటితో చెప్పాల్సిన English script

"One question that changes everything: do you want the full rules – castling, en passant, promotion – or just basic piece movement? Those three need history, not just the current board, so the answer shapes the design.

I'll assume full rules. Before any code, I want to separate three kinds of rule, because they belong in three different places. First, how a piece moves – a bishop goes diagonally. That's the piece's own knowledge. Second, board constraints – something's blocking the path, or the target square has my own piece. That's the piece plus the board. Third, game rules – this move exposes my own king, or castling depends on whether these pieces have moved before. The piece can't know that, and shouldn't.

For the first two, the natural mistake is `canMove(from, to)` returning a boolean. That signature has no board in it, so it can't answer the question. I'd flip it: each piece generates all the squares it can reach, given the board. Sliding pieces share one helper that walks a direction until it hits something – empty square is fine, enemy piece is a capture and then stop, own piece stop. That handles blocking and captures in one place, and the queen becomes rook directions plus bishop directions with no code of its own.

Now the part that actually matters. Those are **pseudo-legal** moves. Consider a white king on e1, a white bishop on e2, and a black rook on e8. The bishop has nine moves by its own rules, and every single one is illegal, because any of them opens the e-file and puts my own king in check. The bishop is pinned, and nothing about the bishop's code is wrong – deciding legality just isn't the piece's job.

So the game filters: make the move, ask whether my king is attacked, unmake it. And 'is the king attacked' reuses the same move generator against the opponent's pieces – no new code. I'd use pseudo-legal moves there deliberately, because a pinned piece still attacks a square even though it can't move.

I'd make and unmake rather than cloning the board. Checking game status generates every legal move for a side, which is hundreds of simulations for one call. If we later add an AI it's millions. Cloning would be the bottleneck.

Castling and en passant need history, which is why pieces carry a `hasMoved` flag and the board carries the square a pawn just skipped over. That square is cleared by every other move – that's the 'only immediately' rule falling out of one line.

One detail I like: checkmate and stalemate are the same check. Any legal moves? Is the king attacked? Those two booleans give all four states, so there's no separate stalemate logic."

17. Follow-ups – undo, AI, performance

Follow-up	జవాబు	మారే classes
"కొత్త పావు (Archbishop)"	<code>class Archbishop extends Piece</code> – bishop dirs + knight steps	+1 కొత్తది, 0 edits
"Undo కావాలి"	<code>#history</code> ఇప్పటికే ఉంది; <code>Move</code> ని ఒక పూర్తి object చేయాలి – కింద చూడండి	Move + Game
"PGN notation"	ఒక వేరే <code>Notation</code> class – core ని తాకదు	+1 కొత్తది
"50-move rule, repetition"	చరిత్రలో position hashes – Zobrist hashing	Game
"వేరే board size (9×9)"	<code>Board</code> lo 8 hardcoded ఉంది → ఒక parameter	Board
"AI ఆడాలి"	కింద చూడండి	0 – design ఇప్పటికే సిద్ధం

Undo

"ఇప్పుడు `#history` lo `{from, to, piece, captured}` మాత్రమే ఉంది – అది undo కి సరిపోదు. Castling అయితే rook కూడా కదిలింది. Promotion అయితే pawn పోయి queen వచ్చింది. En passant అయితే తిన్న pawn వేరే గడిలో ఉంది.

కాబట్టి `Move` ఒక పూర్తి object కావాలి – ఏమి కదిలింది, ఏమి తిన్నది, **ఎక్కడి నుంచి**, ఏ ప్రత్యేక నియమం వర్తించింది, మరియు `hasMoved` ముందు ఏమిటి. అది ఉంటే undo అనేది కేవలం ఆ object ని తిరగబడి అమలు చేయడం – **ఇదే Command pattern**.

గమనించండి – `#simulate` ఇప్పటికే ఈ పని చేస్తోంది, తాత్కాలికంగా. Undo అంటే దాన్ని **శాశ్వతం** చేయడం."

AI

"Minimax కి కావలసినవి రెండే: కదలికలు తయారుచేయడం, మరియు వేగంగా make/unmake. **రెండూ ఇప్పటికే ఉన్నాయి** – `allLegalMoves` మరియు `#simulate`. ఒక evaluation function (పావుల విలువ + స్థానం) చేర్చితే minimax + alpha-beta నేరుగా పైన కూర్చుంటుంది.

ఇది ఒక మంచి సంకేతం: **మనం AI గురించి ఆలోచించకుండానే దానికి సరిపోయే design వచ్చింది**, ఎందుకంటే క్రిగ్ లో copy కి బదులు make/unmake ఎంచుకున్నాం."

Performance

"నా `isAttacked` శత్రు పావులన్నిటి కదలికలనీ తయారుచేస్తుంది – అది వృథా. నిజమైన engines **తిరగబడి** చూస్తాయి: రాజు గడి నుంచి బయటికి కిరణాలు పంపి, అక్కడ శత్రు rook/bishop/queen ఉందా అని చూస్తాయి. అది చాలా వేగవంతం.

అలాగే **bitboards** – 64 గడులు ఒక 64-bit సంఖ్యలో, మరియు కదలికలు bitwise operations. నా design కంటే వందల రెట్లు వేగం. కానీ అది చదవడానికి చాలా కష్టం, మరియు **interview lo స్పష్టత ముఖ్యం**. నేను ఈ రూపాన్నే ఎంచుకుంటాను, మరియు optimisation మార్గాన్ని చెప్తాను."

18. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
తప్పు signature ఒక design ని బంధిస్తుంది	<code>canMove(from,to)</code> (§4)	ఏ API design lo అయినా
"చేయగలదు" ≠ "చేయొచ్చు"	Pseudo-legal vs legal (§6)	Permissions, validation, business rules
నియమాలని పొరలుగా విడదీయడం	పావు / board / ఆట (§1)	Validation layers, middleware
Make / unmake vs copy	<code>#simulate</code> (§8)	Transactions, undo, backtracking, DP
కొన్ని నియమాలకి చరిత్ర కావాలి	Castling, en passant (§9)	Session state, audit, event sourcing
రకం మారడం = వస్తువు మారడం	Promotion (§12)	State pattern, polymorphic replace
రెండు ప్రశ్నలు → నాలుగు స్థితులు	<code>status()</code> (§13)	State machines, truth tables

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess.**

Tic-Tac-Toe తో పోల్చండి ([LLD_Design_Problems_Telugu.pdf](#) Problem 17). అక్కడ కూడా ఒక board, ఒక turn, ఒక win check. కానీ అక్కడ "ఈ కదలిక చెల్లుతుందా?" అనేది **ఒక్క పంక్తి** – ఆ గడి ఖాళీయేనా. ఇక్కడ అది **ఒక మొత్తం పొర**. అదే తేడా ఈ problem ని Medium నుంచి Hard కి తీసుకెళ్తుంది, మరియు అదే interviewer కొలిచేది.

వేగవంతమైన revision కోసం – [LLD_Design_Problems_Telugu.pdf](#) lo Problem 11. Polymorphism పునాది కోసం – [00PS_Telugu.pdf](#) §11. State pattern కోసం – [LLD_Telugu.pdf](#) §28.

Chess – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి board, ప్రతి output నిజంగా `node` lo run చేసి తీసినవే

